

01 · Fundamentos: tipos, operadores y casting.

1° ¿Cuál es la función principal del JDK (Java Development Kit)?

R = D. Ofrecer herramientas necesarias para compilar, depurar y ejecutar aplicaciones Java.

Componente	Función principal
JDK	Desarrollar: compilar, depurar y ejecutar aplicaciones.
JRE	Proporcionar el entorno necesario para ejecutar aplicaciones.
JVM	Ejecutar el bytecode de Java.

2° ¿Qué herramienta se utiliza para empaquetar clases de Java en un archivo JAR (Java Archive)?

R= E. jar

Herramienta	Función
jconsole	Monitorear una aplicación Java: memoria, hilos y otros datos.
java	Iniciar la ejecución de una aplicación Java.
jps	Listar los procesos Java disponibles para la herramienta.
javac	Compilar archivos .java para generar archivos .class.
jar	Crear, consultar y extraer archivos .jar.

3° ¿Cuál de las siguientes herramientas incluidas en el JDK se utiliza para monitorear y gestionar aplicaciones?

R = D. jconsole jconsole es la herramienta gráfica del JDK para monitorear y gestionar aplicaciones Java mediante JMX. Permite observar memoria, hilos y clases cargadas.

4° Which declaration initializes a boolean variable?

R= D

Opción	¿Compila?	Motivo
a. boolean k = 0;	✗	Java no convierte números a boolean.
b. boolean m = null;	✗	boolean es primitivo y no admite null.
c. boolean h = 1;	✗	1 no equivale a true en Java.
d. boolean j = (1 < 5);	✓	La comparación devuelve un valor booleano.

5° ¿Qué declaraciones inicializan correctamente una variable booleana en Java?

R = d. boolean c = (7 >= 7);

e. boolean e = (10 > 5 && 2 < 1);

f. boolean a = (3 < 6);

Opción	¿Compila ?	Explicación
a. boolean d = (4 - 14);	✗	La resta produce -10, un int, no un boolean.
b. boolean f = "true";	✗	Con comillas, "true" es un String.
c. boolean b = (15 = 2);	✗	= es asignación y no se puede asignar un valor al literal 15. Para comparar se usa ==.
d. boolean c = (7 >= 7);	✓	La comparación produce true.
e. boolean e = (10 > 5 && 2 < 1);	✓	Equivale a true && false, que produce false.
f. boolean a = (3 < 6);	✓	La comparación produce true.

6° ¿Cuál es el tipo de resultado cuando se resta un valor short de un valor double en Java?

R = e. double.

Al restar un short de un double, Java promueve automáticamente el short a double antes de realizar la operación.

7° What is the result?

```
public class Main {  
    public static void main(String[] args) {  
        int a = 0;  
        a++;  
        System.out.println(a++);  
        System.out.println(a);  
    }  
}
```

R = b. 1 2.

Línea	Qué ocurre	Valor de a al terminar	Salida
int a = 0;	Inicializa a.	0	—
a++;	Incrementa a; el valor anterior no se utiliza.	1	—
System.out.println(a++);	La expresión entrega 1 e incrementa a a 2. Luego println imprime el 1 recibido.	2	1
System.out.println(a);	Imprime el valor actual.	2	2

8° What is the result?

```
public class Test{  
    public static void main(String[] args){  
        int b = 4;  
        b--;  
        System.out.print(--b);  
        System.out.println(b);  
    }  
}
```

R = c. 22.

Línea	Qué ocurre	Valor de b	Salida
int b = 4;	Inicializa b.	4	—
b--;	Disminuye b en 1.	3	—
System.out.print(--b);	Primero disminuye a 2 y después imprime ese valor.	2	2

<code>System.out.println(b);</code>	Imprime el valor actual: 2.	2	2
-------------------------------------	-----------------------------	---	---

9° ¿Qué imprime este código?

```
public static void main(String[] args) {
    boolean a = true;
    boolean b = false;
    boolean c = a && b || a;
    System.out.println(c);
}
```

R = c. true. true && false produce false. Después, false || true produce true, porque en OR basta con que uno sea verdadero.

10° What is the result?

```
boolean x = true;
boolean y = false;
boolean result = x && y || !x;
System.out.println(result);
```

R = b. false.

1. **x && y → true && false → false.**
2. **Como el lado izquierdo de || es false, se evalúa !x → !true → false.**
3. **false || false → false.**

11° What is the result?

```
public static void main(String[] args) {
    System.out.println("Result:" + 3 + 5);
    System.out.println("Result:" + (3 + 5));
}
```

R = b.

"Result:" + 3 + 5 "Result:3" + 5 "Result:35"	"Result:" + (3 + 5) "Result:" + 8 "Result:8"
--	--

12 ° What is the result?

```
public static void main(String[] args) {
    System.out.println("Result: " + 2 + 3 + 5);
    System.out.println("Result: " + 2 + 3 * 5);
}
```

R = c.

"Result: " + 2 + 3 + 5 "Result: 2" + 3 + 5 "Result: 23" + 5 "Result: 235"	"Result: " + 2 + (3 * 5) "Result: 2" + 15 "Result: 215"
--	---

13°Cuál es el resultado de la operación 9 | 4 en Java?

System.out.println(9 | 4);

R = 13

El resultado binario es 1101: 8 + 4 + 1 = 13.

Valor de la posición	8	4	2	1
9	1	0	0	1
4	0	1	0	0
Resultado	1	1	0	1

14° ¿Cuáles de las siguientes son formas válidas de inicializar una variable booleana en Java?

R = a,b, d.

Opción	¿Compila?	Motivo
a. boolean flag = Boolean.FALSE;	✓	Java convierte automáticamente el objeto Boolean al primitivo boolean: unboxing. Su valor es false.
b. boolean flag = true;	✓	true es un literal booleano.
c. boolean flag = 0;	✗	Java no convierte 0 a boolean.
d. boolean flag = false;	✓	false es un literal booleano.
e. boolean flag = "false";	✗	"false" es un String.

15° ¿Cuáles de los siguientes operadores son válidos para variables booleanas en Java?

R = a, b, d y e.

Operador	¿Válido?	Función y ejemplo
a. &&	✓	AND con cortocircuito: true && false → false.
b.	✓	OR sin cortocircuito: true false → true. Evalúa ambos lados.
c. +	✗	No permite sumar booleanos: true + false no compila.
d.	✓	OR con cortocircuito: true false → true.

e. ==	✓	Compara igualdad: true == false → false.
-------	---	--

16° Which two declarations will compile?

```

14. public static void main(String[] args) {
15.     Int a, b, c = 0;
16.     int a, b, c;
17.     int g, int h, int i = 0;
18.     int d, e, f;
19.     Int k, l, m, = 0;
20. }
```

R = b y d.

Línea	¿Compila?	Explicación
15. Int a, b, c = 0;	✗	Int no es un tipo incorporado en Java. Se usa int o la clase Integer.
16. int a, b, c;	✓	Permite declarar varias variables del mismo tipo separadas por comas.
17. int g, int h, int i = 0;	✗	En una declaración con comas, el tipo se escribe una sola vez.
18. int d, e, f;	✓	Es una declaración válida, aunque todavía no inicializa las variables.
19. Int k, l, m, = 0;	✗	Usa Int y tiene una coma sobrante antes de =.
20. }	—	Cierra el método; no es una declaración de variables.

17° ¿Cuando se asigna una variable byte (no final) a un carácter (char) en Java sin cast explícito, ¿cuál es el tipo de resultado?

R = e. No puedes asignar directamente un byte no final a un char sin cast explícito.

18° What is printed out when the program is executed?

```

public class MainMethod{
    void main(){
        System.out.println("one");
    }
    static void main(String args){
        System.out.println("two");
    }
    public static final void main(String[] args){
        System.out.println("three");
    }
}
```

```

void mina(Object[] args){
    System.out.println("four");
}
}

```

R = b. three.

Método	¿Es el punto de entrada?	Motivo
void main()	✗	No es public static ni recibe String[].
static void main(String args)	✗	Recibe un solo String, no un arreglo; además, no es public.
public static final void main(String[] args)	✓	Cumple los requisitos. final está permitido.
void mina(Object[] args)	✗	Se llama mina, no main, y su firma tampoco corresponde.

19° Which two will compile, and can be run successfully using the following command?

java Fred1 hello walls

R = c y d.

Índice	Valor
args[0]	"hello"
args[1]	"walls"

20° What is the narrowest valid return type for the calculate Method?

```

public class Test {
    ___ doMath (long l, short s) {
        return s / (l * 10);
    }
}

```

R = d. long

Operación	Tipos	Tipo del resultado
l * 10	long * int	long
s / (l * 10)	short / long	long

21° What is the narrowest valid return type for the doMath method?

```
public class Test {  
    ____ doMath (long l, short s) {  
        return s / (l * 10);  
    }  
}
```

R = d. long

22° Dado el siguiente código ¿Cuál es el tipo de retorno válido punto flotante más restringido para el método de cálculo?

```
public class Test {  
    ____ doMath(long l, short s) {  
        return s / (l * 10);  
    }  
}
```

R = b. float.

Paso	Tipo resultante
$l * 10 \rightarrow \text{long} * \text{int}$	long
$s / (l * 10) \rightarrow \text{short} / \text{long}$	long
Retornar ese resultado como float	Conversión automática válida

23° Which of the following lines of code correctly defines the main method in a Java program?

R = c y d.

Opción	¿Válida como punto de entrada?	Motivo
a. static void main(String args[]) {}	✗	Le falta public. La forma String args[] sí es válida.
b. public static int main(String[] args) {}	✗	Debe retornar void. Además, ese cuerpo no compila porque falta devolver un int.
c. public static void main(String... String) {}	✓	String... es varargs, válido en lugar de String[]. El segundo String es el nombre del parámetro.

d. <code>public static void main(String[] args) {}</code>	✓	Es la firma habitual.
e. <code>private static void main(String[] args) {}</code>	✗	Debe ser public, no private.

24° Which of the following statements about the java -jar command, which is used to execute Java programs packaged in JAR (Java Archive) files, are true?

R = b, d y e.

Opción	¿Correcta?	Explicación
a	✗	Con -jar, las opciones -cp y -classpath no se usan para agregar bibliotecas.
b	✓	Se ignoran la variable de entorno CLASSPATH y las opciones de classpath de la línea de comandos.
c	✗	Tener varias clases con main no permite que Java elija automáticamente. El manifiesto debe indicar cuál iniciar.
d	✓	El JAR necesita un manifiesto que especifique la clase principal.
e	✓	Describe el requisito de un JAR ejecutable: el manifiesto con el atributo Main-Class.

02 · Control de flujo.

1° ¿Cuál de las siguientes afirmaciones sobre el ciclo for mejorado introducido en Java 5 es verdadera?

R = b.

Para tus notas: El for-each recorre arreglos y objetos Iterable. Reasignar la variable del ciclo no reemplaza el elemento original; sí se puede modificar el estado de un objeto mutable mediante su referencia.

2° What is the result value of w?

R = c: 0.

Instrucción	Valores de las variables	Salida
int a = 10;	a = 10	—
int b = 37;	b = 37	—
int z = 0;	z = 0	—
int w = 0;	w = 0	—
if (a == b)	10 == 37 → false; se omite z = 3;	—
else if (a > b)	10 > 37 → false; se omite z = 6;	—
w = 10 * z;	w = 10 * 0 → 0	—
System.out.println("El valor de w es: " + w);	a = 10, b = 37, z = 0, w = 0	El valor de w es: 0

3° ¿Que muestra en pantalla el siguiente código?

R = b: Count: 5.

Instrucción	Valores de las variables	Salida
int count = 0; e int i = 0;	count = 0, i = 0	—
i % 2 == 0 → true; count++;	i = 0, count: 0 → 1	—
i++; y evaluar el if → false	i = 1, count = 1	—
i++; y evaluar el if → true; count++;	i = 2, count: 1 → 2	—
i++; y evaluar el if → false	i = 3, count = 2	—

i++; y evaluar el if → true; count++;	i = 4, count: 2 → 3	—
i++; y evaluar el if → false	i = 5, count = 3	—
i++; y evaluar el if → true; count++;	i = 6, count: 3 → 4	—
i++; y evaluar el if → false	i = 7, count = 4	—
i++; y evaluar el if → true; count++;	i = 8, count: 4 → 5	—
i++; y evaluar el if → false	i = 9, count = 5	—
i++; y evaluar i < 10 → false	i = 10; termina el ciclo, count = 5	—
System.out.println("Count: " + count);	count = 5; i ya está fuera de alcance	Count: 5

4° ¿Cuál es la salida del programa?

R = a.

En ciclos anidados, el ciclo interno se completa por cada iteración del externo. Su inicialización se ejecuta nuevamente cada vez que se entra en él. Aquí se imprimen $3 \times 2 = 6$ líneas.

5° ¿Cuántas veces se ejecutará la instrucción println más interna en el siguiente código?

R = c, 24 veces.

- i: 0, 1, 2, 3 → 4 iteraciones.
- j: 0, 1, 2 → 3 iteraciones por cada i.
- k: 0, 1 → 2 iteraciones por cada combinación de i y j.

Por lo tanto, println se ejecuta $4 \times 3 \times 2 = 24$ veces.

6° What is the result?

```
public class DoWhile {
    public static void main(String[] args) {
        int ii = 2;
        do {
            System.out.println(ii);
        } while (ii > 0);
    }
}
```

R = d: An infinite loop.

do...while ejecuta primero el cuerpo y después evalúa la condición, por lo que se ejecuta al menos una vez. Si la condición permanece verdadera y no hay una salida, el ciclo es infinito.

7° What is the result?

```
public class DoWhile {
    public static void main(String[] args){
        int ii = 2;
        do{
            System.out.println(ii);
        }while(--ii);
    }
}
```

R = b

Instrucción	Análisis de tipos y valores	Salida
int ii = 2;	Declaración válida de una variable int	No se ejecuta
System.out.println(ii);	Instrucción válida por sí misma	No se ejecuta
while (--ii);	--ii tiene tipo int; se requiere una condición booleana	Error de compilación

8° What value should replace kk in line x to cause jj=5 to be output?

R = e: 11

Un for comprueba la condición antes de cada iteración. Desde 11, con $ii > 6$ y decrementos de uno, hay cinco iteraciones. Los operadores $-=$, $++$ y $--$ incluyen la conversión necesaria al tipo de su variable.

9° ¿Qué imprime este bucle?

R = b: 3.

Los índices de un arreglo comienzan en 0. $i \% 2 == 0$ comprueba si el índice es par; $arr[i] \% 2 == 0$ comprueba si el elemento es par. $num += arr[i]$ acumula una suma y $num -= arr[i]$ acumula una resta.

10° What is the result if the integer value is 33?

R = a. The fox lazy....

Instrucción	Valores de las variables y evaluación	Salida
int value = 33;	value = 33	—
if (value >= 0)	$33 \geq 0 \rightarrow \text{true}$; entra	—
if (value != 0)	$33 \neq 0 \rightarrow \text{true}$	the
else de if (value != 0)	Se omite	—
if (value < 10)	$33 < 10 \rightarrow \text{false}$	—

if (value > 30)	33 > 30 → true	fox
else if (value < 50)	Se omite porque ya se ejecutó una rama	—
else if (value < 10)	Se omite	—
else	Se omite	—
if (value > 10)	33 > 10 → true	lazy
else de if (value > 10)	Se omite	—
System.out.println("...");	value = 33	...

11° What is the result?

R = c.

En un switch con etiquetas case ...;, la ejecución comienza en el caso coincidente y continúa hasta un break u otra salida, o hasta el final del switch.

12° What is the result?

R = e.

Instrucción	Valores de las variables	Salida
int y = 1;	y = 1	—
Days d = Days.TUESDAY;	d = TUESDAY, y = 1	—
switch (d)	Coincide con case TUESDAY; omite MONDAY	—
y += 10;	y = 1 + 10 = 11	—
y += 100;	y = 11 + 100 = 111	—
y += 1000;	y = 111 + 1000 = 1111	—
System.out.println(y);	y = 1111	1111

13° Que se imprime en pantalla al ejecutar el siguiente código?

R = f. BCF.

Instrucción	Valores de las variables y ejecución	Salida acumulada
final char a = 'A', d = 'D';	a = 'A', d = 'D'	—

char grade = 'B';	grade = 'B'	—
switch (grade)	Entra directamente en case 'B'	—
System.out.print("B");	Imprime; no hay break	B
System.out.print("C");	Continúa por fall-through	BC
case d: y case 'F':	Son etiquetas; no detienen la ejecución	BC
System.out.print("F");	Imprime y termina el switch	BCF

14° What is the result?

R = f: BC.

El break termina el switch cuando la ejecución llega a él, incluso si se entró por un caso anterior. Aquí se entra por 'B', se continúa hacia 'C' y su break impide llegar a los casos siguientes.

15° ¿Qué muestra en pantalla el siguiente código?

R = b:greater than - 6

Instrucción	Valores de las variables	Salida
int x = 0;	x = 0	—
x++ < 5 → 0 < 5 → true	x: 0 → 1; entra al cuerpo vacío	—
x++ < 5 → 1 < 5 → true	x: 1 → 2; entra	—
x++ < 5 → 2 < 5 → true	x: 2 → 3; entra	—
x++ < 5 → 3 < 5 → true	x: 3 → 4; entra	—
x++ < 5 → 4 < 5 → true	x: 4 → 5; entra	—
x++ < 5 → 5 < 5 → false	x: 5 → 6; termina el ciclo	—
String message = x > 5 ? "greater than" : "false";	6 > 5 → true; message = "greater than"	—
System.out.println(message + " - " + x);	message = "greater than", x = 6	greater than - 6

16° How many times is 2 printed?**R = e. Once**

Instrucción	Valores de las variables	Salida
Inicializar table e ii	table.length = 3, ii = 0	—
Primera iteración del for	ss = "aa", ii = 0	—
while (ii < table.length)	0 < 3 → true	—
System.out.println(ii);	ii = 0	0
ii++; y break;	ii = 1; sale del while	—
Segunda iteración del for	ss = "bb", ii = 1	—
while (ii < table.length)	1 < 3 → true	—
System.out.println(ii);	ii = 1	1
ii++; y break;	ii = 2; sale del while	—
Tercera iteración del for	ss = "cc", ii = 2	—
while (ii < table.length)	2 < 3 → true	—
System.out.println(ii);	ii = 2	2
ii++; y break;	ii = 3; sale del while	—
Termina el for	No quedan elementos en table	—

17° ¿Cuál es el resultado?**R = d. x = 4.**

Instrucción	Valores de las variables y evaluación	Salida
static boolean b;	b = false por defecto	—
int x = 0;	x = 0, b = false	—
if (b)	false; omite x = 1;	—
else if (b = false)	Asigna false a b; la expresión vale false; omite x = 2;	—
else if (b)	b = false; omite x = 3;	—

else { x = 4; }	x = 4, b = false	—
System.out.println("x=" + x);	x = 4	x=4

18° Which one is valid as a replacement for foo?

R = d. "foo".equals("bar").

El tipo del resultado completo determina si una expresión sirve como condición. equals() devuelve boolean; compareTo() devuelve int. Un ternario que elige entre dos enteros produce un entero, aunque su condición interna sea booleana. La estructura exige el tipo: while necesita una condición **booleana**.

19° ¿Que imprime en pantalla el código anterior?

R = a. 12432432 The co value = 8.

Instrucción	Valores de las variables y evaluación	Salida acumulada
Inicializar co y x	co = 0, x = 1	—
Inicialización: test('1')	co = 1; devuelve true, que se descarta	1
Condición: test('2') && (x <= 2)	co = 2; true && (1 <= 2) → true	12
Cuerpo: x++;	x = 2, co = 2	12
Cuerpo: test('4')	x = 2, co = 3	124
Actualización: test('3')	x = 2, co = 4	1243
Condición: test('2') && (x <= 2)	co = 5; true && (2 <= 2) → true	12432
Cuerpo: x++;	x = 3, co = 5	12432
Cuerpo: test('4')	x = 3, co = 6	124324
Actualización: test('3')	x = 3, co = 7	1243243
Condición: test('2') && (x <= 2)	co = 8; true && (3 <= 2) → false; termina	12432432
System.out.println(" The co value = " + co);	x = 3, co = 8	12432432 The co value = 8

La actualización del for va después del cuerpo y antes de comprobar nuevamente la condición. Si la condición da false, se termina el ciclo sin ejecutar otra actualización.

20° What values of x, y, z will produce the following result?

R = d. int x = 2, y = 3, z = 4;

- x = 2: el for ejecuta dos iteraciones y cada una termina imprimiendo ---.
- y = 3: en la primera iteración del for, el do...while imprime tres filas.
- z = 4: el while interno imprime los números del 1 al 4 en cada fila.

La clave es que j no se reinicia. En la segunda iteración del for, comienza en 3. Sin embargo, el do...while ejecuta su cuerpo una vez antes de comprobar la condición.

03 · Strings y clases envoltorio

1° Which of the following lines of code correctly appends a string to a StringBuilder object in Java?

R = b.`StringBuilder sb = new StringBuilder("Hello"); sb.append(" World");`

StringBuilder es mutable: append(texto) agrega texto al final del mismo objeto. insert(posición, texto) agrega texto en una posición determinada.

2° ¿Cuál de las siguientes líneas de código utiliza correctamente un método de la clase String para reemplazar todas las apariciones del carácter 'a' por 'e' en una cadena dada en Java?

R = a.`String result = "banana".replace('a', 'e');`

cadena.replace('a', 'e') reemplaza todas las 'a' por 'e'. Como String es inmutable, debes guardar o utilizar el valor devuelto.

3° What is the best way to test that the values of h1 and h2 are the same?

R = a.`if (h1.equals(h2)).`

En String, equals() compara contenido y == compara referencias. Dos objetos distintos pueden contener el mismo texto: en este ejemplo, equals() devuelve true y == devuelve false.

4° What is the result?

R = c.`A StringIndexOutOfBoundsException is thrown at runtime.`

charAt(i) devuelve el carácter del índice i. Los índices válidos van de **0 a length() - 1**. Usar charAt(length()) compila, pero lanza StringIndexOutOfBoundsException en ejecución.

5° What is the result?

R = d.`message`

String es inmutable. s.toUpperCase() devuelve una cadena en mayúsculas, pero aquí no se guarda ni se utiliza ese resultado. La cadena original permanece igual.

6° What is the result?

R = a.`ticgameTAC`

Instrucción	Valores de las variables / resultado	Salida
String a = "tic";	a = "tic"	Ninguna
String b = "tac";	b = "tac"	Ninguna

a.substring(0, 2);	Devuelve "ti": incluye el índice 0 y excluye el 2. Se descarta; a = "tic"	Ninguna
a.concat("toc");	Devuelve "tictoc". Se descarta; a = "tic"	Ninguna
b.toUpperCase()	Devuelve "TAC", que se usa en la concatenación. b sigue siendo "tac"	Ninguna
"game" + b.toUpperCase()	Produce "gameTAC"	Ninguna
a += "game" + b.toUpperCase()	Se reasigna a con "ticgameTAC". La expresión también entrega ese valor	Ninguna
System.out.println(...)	a = "ticgameTAC", b = "tac"	ticgameTAC

7° What will be the output of the following code snippet?

R = c.true true false false

En String, equals() compara **contenido** y == compara **referencias**. Construir el mismo texto mediante +, concat() o StringBuilder no implica obtener el mismo objeto. StringBuilder.toString() devuelve un String con el contenido del builder.

8° ¿Qué se imprime en pantalla al ejecutar el siguiente código?

R = a.false true false

intern() busca por contenido y devuelve la referencia del pool. En new String("example"), el objeto nuevo es distinto del literal del pool; por eso str1 == str1.intern() es false.

9° Given the following code snippet, what will be the output of the program?

R = d.true false true

Los literales String con el mismo contenido comparten un objeto del pool. new String(...) crea un objeto distinto. intern() devuelve la referencia del pool: aquí s1, s3 y s4 apuntan al mismo objeto, mientras que s2 apunta a otro.

10° ¿Qué imprime en pantalla este código?

R = a.1 2 3 4 5 6

Instrucción	Valores de las variables	Salida de esa instrucción
String str = "a1b2c3d4e5f6";	str = "a1b2c3d4e5f6"	Ninguna
String[] parts = str.split("\\D");	parts = ["", "1", "2", "3", "4", "5", "6"]	Ninguna
Primera iteración: System.out.print(part + " ");	part = ""	" "
Segunda iteración	part = "1"	"1 "
Tercera iteración	part = "2"	"2 "
Cuarta iteración	part = "3"	"3 "
Quinta iteración	part = "4"	"4 "
Sexta iteración	part = "5"	"5 "
Séptima iteración	part = "6"	"6 "

11° What is the result?**R = c.A NullPointerException occurs at runtime**

Instrucción	Valores de las variables / resultado	Salida
<code>new Integer(4)</code>	Crea un Integer con valor 4	Ninguna
<code>new Boxer1(...)</code>	Hace unboxing del argumento para el parámetro int y: <code>y = 4</code>	Ninguna
Inicialización de los campos	<code>i = null, x = 0</code>	Ninguna
<code>x = i + y;</code>	Intenta hacer unboxing de <code>i</code> : lanza <code>NullPointerException</code> . La asignación no se completa; <code>x</code> sigue en 0	Ninguna
<code>System.out.println(x);</code>	No se ejecuta	Ninguna

Los campos Integer tienen valor predeterminado null; los campos int, 0. Usar un Integer nulo en una suma provoca `NullPointerException` por unboxing, aunque el código compile. Las variables locales no reciben estos valores predeterminados.

12° What is the result?**R = c.Hello Heo dlroW olleH**

Instrucción	Valores / cambios	Salida
<code>String str = "Hello";</code>	<code>str = "Hello"</code>	Ninguna
<code>modify(str);</code>	Dentro del método, <code>s</code> se reasigna a "Hello World"; <code>str</code> permanece igual	Ninguna

System.out.println(str);	str = "Hello"	Hello
StringBuffer sbf = new StringBuffer("Hello");	Contenido de sbf: "Hello"	Ninguna
modify(sbf);	append() cambia el contenido a "Hello World"	Ninguna
System.out.println(sbf.replace(2, sbf.length(), "o"));	sbf.length() es 11; replace(2, 11, "o") modifica el contenido a "Heo"	Heo
StringBuilder sbd = new StringBuilder("Hello");	Contenido de sbd: "Hello"	Ninguna
modify(sbd);	append() cambia el contenido a "Hello World"	Ninguna
System.out.println(sbd.reverse());	reverse() modifica el contenido a "dlroW olleH"	dlroW olleH

String es inmutable; StringBuffer y StringBuilder son mutables. Java pasa las referencias por valor: reasignar el parámetro no cambia la variable original, pero modificar el objeto compartido sí es visible fuera del método. replace(inicio, fin, texto) excluye fin; reverse() invierte el contenido.

04 · Arreglos

1° Which of the following are valid ways to initialize an array in Java?

R = b.int[] arr = {1, 2, 3};

c.int[] arr = new int[] {1, 2, 3};

d.int[] arr = new int[3];

Un arreglo puede inicializarse con {1, 2, 3}, con new int[] {1, 2, 3} o indicando su tamaño: new int[3]. En este último caso, sus elementos reciben valores predeterminados; para int, es 0.

2° ¿Qué imprime este código de matriz?

R = d.15

array[filas][columna] obtiene un elemento. Usar array[i][i] recorre la diagonal principal. result += valor equivale aquí a result = result + valor: acumula, no sustituye lo anterior.

En un arreglo bidimensional, array.length cuenta las filas y array[i].length cuenta los elementos de la fila i. Se escribe **length sin paréntesis**.

3° What is the result?

R = c.159

En ciclos anidados, el ciclo interno termina antes de incrementar la variable del externo.

Aquí, j se reinicia en 0 para cada fila. i == j selecciona la diagonal principal; matrix[i][j] + " " concatena el elemento con un espacio.

4° What is the result?

R = 4 An ArrayIndexOutOfBoundsException is thrown at run time.

En array[filas][columna], valida cada índice por separado: fila < array.length y columna < array[filas].length, ambos mayores o iguales a 0. Las filas pueden tener distintas longitudes.

Un índice fuera de rango provoca **ArrayIndexOutOfBoundsException en ejecución**, no devuelve null.

5° What is the result?

R = b.3true1

array2D.length cuenta filas; array2D[i].length cuenta elementos de esa fila. Cada fila es un arreglo, por lo que array2D[i].getClass().isArray() devuelve true si la fila no es null. + "" convierte el resultado en un String sin añadir espacios.

6° ¿Qué imprime este código?

R = a.1

Arrays.sort(arreglo) modifica el orden del arreglo. Arrays.binarySearch(arreglo, valor) requiere que esté ordenado según el mismo criterio y devuelve el índice si encuentra el valor. Si no lo encuentra, devuelve -(puntoDeInserción) - 1, no necesariamente -1.

7° ¿Que imprime en pantalla el código anterior?

R = b.[2, 1, 3, 4]

Arrays.sort(arreglo, inicio, fin) ordena desde inicio incluido hasta fin excluido. El límite fin puede ser igual a arreglo.length. Los elementos fuera del rango conservan sus posiciones.

8° What is the result?

R = e.35

System.arraycopy(origen, inicioOrigen, destino, inicioDestino, cantidad) copia elementos y sobrescribe las posiciones de destino, sin cambiar la longitud del arreglo. El último argumento indica cuántos elementos copiar. Origen y destino pueden ser el mismo arreglo.

9° Which two are valid instantiations and initializations of a multidimensional array?

R = a.int[][] array2D = new int[2][2]; array2D[0][0] = 1; array2D[0][1] = 2; array2D[1][0] = 3; array2D[1][1] = 4;

b.int[] array = {0, 1}; int[][][] array3D = new int[2][2][2]; array3D[0][0] = array; array3D[0][1] = array; array3D[1][0] = array; array3D[1][1] = array;

Cada acceso con [índice] reduce una dimensión: $\text{int}[][][] \rightarrow \text{int}[][] \rightarrow \text{int}[] \rightarrow \text{int}$. Se puede asignar un $\text{int}[]$ a una posición de tipo $\text{int}[]$, compartiendo el mismo objeto. Repetir asignaciones y tener filas de distintas longitudes es válido; los inicializadores deben respetar los tipos y separar sus elementos con comas.

05 · Clases, métodos y constructores.

1° En Java, ¿qué sintaxis es usada para declarar un parámetro en un método que acepte un número variable de argumentos?

R = b....

Varargs se declara con ... después del tipo, por ejemplo, int... numeros. Acepta cero o más argumentos y se maneja como un arreglo dentro del método. Solo puede haber uno y debe ser el último parámetro.

2° En Java, ¿cuál de las siguientes se utiliza para llamar a un constructor desde otro constructor dentro de la misma clase?

R = d.this()

Opción	¿Correcta?	Explicación
a. super()	No	Llama a un constructor de la superclase.
b. new()	No	No es una sintaxis válida. Para crear un objeto se utiliza new NombreClase(...).
c. this.<class_name>() ()	No	No es la sintaxis para invocar un constructor.
d. this()	Sí	Invoca otro constructor de la misma clase, en este caso sin argumentos.

3° Which class has a default constructor?

R =g. X only.

Un default constructor (constructor predeterminado) es el que genera automáticamente el compilador cuando una clase no declara ningún constructor.

Clase	¿Tiene constructor predeterminado?	Explicación
X	Sí	No declara constructores; el compilador genera X().
Y	No	Y(){ } es un constructor sin argumentos declarado explícitamente, no uno predeterminado.
Z	No	Declara Z(int i), por lo que el compilador no genera un constructor sin argumentos.

4° A method is declared to take three arguments. A program calls this method and passes only two arguments. What is the result?

R = c.Compilation fails

Los parámetros obligatorios requieren argumentos: Java no rellena los faltantes. Si no existe una firma compatible, hay error de compilación. Un parámetro varargs puede recibir cero elementos.

5° ¿Cuál de las siguientes afirmaciones describe correctamente un objeto inmutable en Java?

R = d.Un objeto inmutable no permite ningún cambio en su estado después de ser creado.

Un objeto inmutable no cambia su estado después de crearse. String es inmutable; StringBuilder y StringBuffer son mutables. Reasignar una variable no modifica el objeto al que apuntaba.

6° What is the result?

R = a.hello

Instrucción o etapa	Valores de las variables	Salida
Se reserva el objeto	str = null, valor predeterminado del campo	—
String str = "default";	str = "default"	—
Se ejecuta el cuerpo de X(String s)	El parámetro s = "hello"; str = "default"	—
str = s;	str cambia a "hello"	—
Termina la construcción y se llama a print()	str = "hello"	—
System.out.println(str);	Sin cambios	hello y un salto de línea

7° What is the result?

R = b.12

Java siempre pasa argumentos por valor. Para un int, el método recibe una copia del número. Modificar el parámetro no modifica la variable original, aunque tengan el mismo nombre.

8° Dado: ¿Cuál es el resultado?

R = c.20

Cada llamada crea un nuevo parámetro con una copia del argumento. Aquí, ambas reciben 20 y lo cambian localmente a 30; la variable de main sigue en 20. Un método static puede invocarse mediante NombreClase.metodo().

9° What is the result?

R = d.Display with no arguments

Display with integer: 5

Display with string: Hello

Instrucción	Método seleccionado y valor del parámetro	Salida
OverloadedStaticMethods.display();	display(): sin parámetros	Display with no arguments
OverloadedStaticMethods.display(5);	display(int num): num = 5	Display with integer: 5
OverloadedStaticMethods.display("Hello");	display(String str): str = "Hello"	Display with string: Hello

Los métodos static sí pueden sobrecargarse. La sobrecarga requiere listas de parámetros diferentes por cantidad, tipos u orden de tipos. El compilador elige la versión según los argumentos; cambiar solo el tipo de retorno no basta.

10° What will make this code compile and run?

R = e.Change line 7 to the following: price.price = 4;

Instrucción	Valores de las variables	Salida
Simple price = new Simple();	La variable local referencia el nuevo objeto. Su campo price vale 0.0f por defecto.	—
price.price = 4;	La referencia local no cambia. El campo del objeto pasa de 0.0f a 4.0f.	—
Termina main	Sin más cambios	No se imprime nada

11° What is the result when this program is executed?

R = b.Bob's Name: Jian

Asignar una referencia no copia el objeto. Después de jian = bob, ambas variables referencian el mismo objeto: modificar jian.name se observa también mediante bob.name.

12° What is the result?

R = b.Compilation fails

Varargs (...) debe ser el último parámetro y solo puede haber uno por método. go(String... y, int x) no compila; go(int x, String... y) sí. Dentro del método, y se utiliza como un arreglo.

13° ¿Cuál es el resultado?

R = b.Hello

Reasignar un parámetro String no reasigna la variable original. `s += "Java"` cambia la referencia local de `s`, no el objeto compartido. `main(String... String)` es válido: String también puede usarse como nombre de variable.

14° ¿Cuáles dos opciones son verdaderas sobre el constructor por defecto?

R = a. El constructor por defecto siempre invoca el constructor sin argumentos de la superclase.

d. El constructor por defecto se crea solo cuando la clase no tiene ningún constructor definido.

El constructor por defecto se genera únicamente si no declaras ningún constructor, tiene el mismo acceso que la clase e invoca `super()` —excepto en `Object`, que no tiene superclase—. **No siempre es público y los constructores no se heredan.**

15° Which three options correctly describe the relationship between the classes?

R = c. Class2 has-a v2.

f. Class2 has-a Class1.

has-a describe una relación mediante campos y tiene dirección: si `Class2` declara `Class1 c1`;, entonces `Class2` has-a `Class1`. Los campos del objeto referenciado no se convierten en campos propios.

16° What is the result?

R = e.6564

Instrucción	z de main	z de doStuff	Campo myScope.z	Salida acumulada
<code>ScopeTest myScope = new ScopeTest();</code>	—	—	0	—
<code>int z = 6;</code>	6	—	0	—
<code>System.out.print(z);</code>	6	—	0	6
<code>myScope.doStuff(); → int z = 5;</code>	6	5	0	6
<code>doStuff2(); → z = 4;</code>	6	5	4	6
Regresa a <code>doStuff()</code> : <code>System.out.print(z);</code>	6	5	4	65
Regresa a <code>main</code> : <code>System.out.print(z);</code>	6	Ya no está disponible	4	656
<code>System.out.print(myScope.z);</code>	6	—	4	6564

Las variables locales solo son accesibles dentro de su ámbito; no se comparten con los métodos que llaman. En `doStuff2()`, `z = 4` modifica el campo porque no existe una variable local `z`. Salida: 6564.

17° What is the result?

R = a.Compilation fails

No compila. Falta = y myScope solo existe dentro de main. En un método de instancia, this.z accede al campo, incluso si una variable local z lo oculta. Revisa también println() frente a print().

18° What is the result?

R = d.Constructor with argument called: Default No-argument constructor called Name: Default Hello:Default Constructor with argument called: John Name: John Hi:John

this(...) ejecuta otro constructor de la misma clase antes de continuar con las instrucciones restantes. La sobrecarga selecciona el constructor o método según los argumentos. Aquí, primero se imprime el mensaje del constructor con "Default".

19° What is the result?

R = c.Constructor with int argument: 10 No-argument constructor No-argument calculate method Calculate method with int argument: 5 Constructor with int argument: 20 No-argument calculate method Calculate method with int argument: 15

Instrucción	Variables y seguimiento	Salida
new OverloadedMethods() → this(10);	Se ejecuta el constructor con x = 10	Constructor with int argument: 10
Continúa el constructor sin argumentos	Termina la llamada a this(10)	No-argument constructor
obj1.calculate();	Se selecciona el método sin parámetros	No-argument calculate method
obj1.calculate(5);	El parámetro de esta llamada es x = 5	Calculate method with int argument: 5
new OverloadedMethods(20);	Se llama directamente al constructor con x = 20	Constructor with int argument: 20
obj2.calculate();	Se selecciona el método sin parámetros	No-argument calculate method
obj2.calculate(15);	El parámetro de esta llamada es x = 15	Calculate method with int argument: 15

this(10) ejecuta primero el constructor con int y después continúa el constructor que lo llamó. La sobrecarga selecciona una versión según los argumentos; no ejecuta todas las versiones. Después, sigue las llamadas de main en orden.

20° ¿Cuál será el resultado del siguiente fragmento de código?

R = c. Constructor with name and age called

Default constructor called

Constructor with name and age called

Constructor with name called

Constructor with name and age called

Instrucción	Valores de los campos del objeto en construcción	Salida
Se crea obj1 y el constructor sin argumentos ejecuta this("Unknown", 0)	Inicialmente: name = null, age = 0	—
El constructor de dos parámetros asigna this.name = name; y this.age = age;	name = "Unknown", age = 0	Constructor with name and age called
Regresa al constructor sin argumentos	Sin cambios	Default constructor called
Se crea obj2 y el constructor con nombre ejecuta this(name, 0)	Inicialmente: name = null, age = 0; se pasan "Alice" y 0	—
El constructor de dos parámetros asigna los campos	name = "Alice", age = 0	Constructor with name and age called
Regresa al constructor con nombre	Sin cambios	Constructor with name called
Se crea obj3 directamente con el constructor de dos parámetros y se asignan los campos	name pasa de null a "Bob"; age pasa de 0 a 30	Constructor with name and age called

Al encontrar this(...), sigue primero el constructor al que llama y después regresa a la siguiente instrucción. Aquí: obj1 imprime dos líneas, obj2 imprime dos y obj3 imprime una.

21° Dado el siguiente código ¿cuál es el resultado?

R = a.Error de compilación

Un ciclo entre constructores mediante this(...) causa error de compilación, no StackOverflowError. La recursión en métodos sí está permitida, pero sin una condición de salida puede agotar la pila en ejecución.

22° What will be the result of attempting to compile and run the following code snippet?

R = a. Compilation error due to recursive constructor calls.

Las llamadas entre constructores mediante this(...) no pueden formar un ciclo. Si un constructor termina delegando en sí mismo, directa o indirectamente, hay error de compilación, no un error en ejecución.

23° Give What is the result?

R = f.double

Método	Conversión necesaria	Prioridad
dolt(double d)	Ampliación primitiva: float → double	Primera: se selecciona
dolt(Float f)	Autoboxing: float → Float	Posterior
dolt(float... fs)	Agrupar el argumento en un arreglo para varargs	Última

Para este tipo de sobrecarga: coincidencia exacta → ampliación primitiva → autoboxing → varargs. Con un argumento float, dolt(double) gana frente a dolt(Float) y dolt(float...).

24° ¿Cuántos de los fragmentos de código, si compilan?

R = b.3

Fragmento	¿Compila?	Explicación
public static void main(String...a) {	Sí	Declara un varargs de String. No es obligatorio dejar un espacio después de
public static void main(String. a) {	No	Un solo punto no es una sintaxis válida para declarar este parámetro.
public static void main(String... a) {	Sí	Es la declaración habitual de un varargs de String.
public static void main(String[]... a) {	Sí	Declara un varargs cuyos elementos son arreglos String[]. Dentro del método, a es de tipo String[][].
public static void main(String...[] a) {	No	Los corchetes no pueden colocarse así después de La forma válida para ese varargs es String[]... a.

06 · Modificadores, static e inicialización

1° ¿Qué paquete está importado por defecto en Java?

R = b. java.lang

java.lang se importa automáticamente en Java. Incluye String, System, Object y las clases envoltorio. Los otros paquetes de las opciones requieren un import o usar el nombre completo de la clase.

2° ¿Cuál de los siguientes paquetes en Java no necesitan ser importados explícitamente en un programa Java?

R = b. java.lang

Java importa implícitamente java.lang. Los demás paquetes de la pregunta no se importan automáticamente; sus clases pueden usarse mediante un import o su nombre completo.

3° ¿Qué ocurre cuando ejecutas este fragmento de código?

R = b. Imprime "Resultado: 15".

Instrucción	Valores de las variables	Salida
Entra a main	args: argumentos del programa	—
Llama a Utilidades.sumar(5, 10)	Se inicializan a = 5, b = 10	—
Ejecuta return a + b;	a y b no cambian; devuelve 15	—
Regresa a main y continúa la concatenación	Se obtiene "Resultado: 15"	—
Ejecuta System.out.println(...)	Sin cambios	Resultado: 15 y un salto de línea

Un método static puede llamarse con Clase.metodo(), sin crear un objeto.

Utilidades.sumar(5, 10) devuelve 15; println imprime el texto y añade un salto de línea.

4° What is a static block of code in Java?]

R = b. A block of code inside a class that runs when that class first loaded in JVM

Un bloque static se ejecuta una sola vez al inicializar la clase, no cada vez que creas un objeto.

5° We have the following Java class:

R = a. From the class, the package, subclasses.

protected permite acceso desde la misma clase, el mismo paquete y las subclasses. Fuera del paquete, una subclase puede acceder al campo mediante una referencia de su propio tipo o de un subtipo suyo; no mediante una referencia cualquiera de la superclase.

6° What two keywords may precede the word 'class' in a class declaration?

R = c. Static, e.Public

public modifica a Externa.

static modifica a la clase miembro Interna.

7° ¿Cuáles son los modificadores que pueden preceder a una declaración de clase de nivel superior?

R = b. abstract, c. strictfp, f.public, h.final.

Una clase de nivel superior admite abstract, strictfp, public y final; no admite protected, private ni static.

8° What changes will make this code compile?

R = c.Changing the private modifier on the declaration of the one() method to protected.

Una subclase no puede acceder directamente a métodos private de su superclase; cambiar one() a protected permite la llamada. main sí puede llamar a un método privado de su propia clase mediante un objeto: new Y().two()

9° Which statement is true?

R = a.420 is the output.

Instrucción	Valores / estado	Salida
new ExtendedA(420)	Se crea un objeto. Su campo numberOfInstances empieza con el valor predeterminado 0.	—
Entra a ExtendedA(int numberOfInstances)	El parámetro del constructor vale 420.	—
super(numberOfInstances)	Llama a otro constructor: ClassA(420). Su parámetro también vale 420.	—
super() implícito en ClassA	Entra a Object(), termina y regresa a ClassA.	—
this.numberOfInstances = numberOfInstances;	El campo del objeto cambia de 0 a 420. El parámetro si	

Los constructores pueden ser public, protected, private o tener acceso de paquete. Aquí main puede llamar al constructor privado de su propia clase, y super(420) llama al constructor protegido de la superclase. this.numberOfInstances identifica el campo; numberOfInstances identifica el parámetro.

10° Dado el siguiente código. ¿Cuál es el resultado?

R = b.static default Constructor Variable

Instrucción	Valores / estado	Salida
static String saludo = "Variable";	Antes de entrar a main, saludo pasa de su valor predeterminado null a "Variable".	—
Bloque static	saludo = "Variable".	static
Entra a main y ejecuta new Prioridad()	Comienza la construcción de un objeto. La inicialización estática ya terminó.	—
super() implícito	Llama a Object() y regresa.	—
Bloque de instancia	Continúa la inicialización del objeto, antes del cuerpo del constructor.	default
Cuerpo de Prioridad()	Entra al cuerpo y termina la construcción.	Constructor
Se completa Prioridad p = new Prioridad();	p referencia al objeto creado.	—
p.getSaludo();	Entra al método y lee saludo, que sigue siendo "Variable".	Variable

11° ¿Cuál es el resultado al ejecutar el siguiente código?

R = c.Static block Static method Instance block Constructor

Instrucción	Valores / estado	Salida
Bloque static	Se inicializa la clase antes de entrar a main. No hay objetos creados.	Static block
StaticNonStaticBlockTest.staticMethod();	Entra al método estático, imprime y regresa a main. No necesita un objeto.	Static method
new StaticNonStaticBlockTest();	Continúa main: comienza la construcción de un objeto.	—
super() implícito	Llama al constructor de Object, termina y regresa.	—
Bloque de instancia	Se ejecuta antes del cuerpo del constructor.	Instance block

Cuerpo de StaticNonStaticBlockTest()	Imprime y termina la construcción. Se ha creado un objeto.	Constructor
--------------------------------------	--	-------------

12° What is the result?

R = d.ABCX

Instrucción	Valores de las variables / referencias	Salida
new Test("M")	Comienza la construcción del objeto 1. Entra a Test(String s) con s = "M". El campo value empieza en null.	—
super() implícito	Llama a Object(), regresa y continúa la inicialización del objeto 1.	—
String value = "A";	Objeto 1: value cambia de null a "A".	—
Primer bloque: value += "B";	Objeto 1: "A" → "AB".	—
Segundo bloque: value += "C";	Objeto 1: "AB" → "ABC".	—
Cuerpo de Test(String s): value += s;	Objeto 1: "ABC" → "ABCM".	—
Se completa Test m = new Test("M");	Regresa a main. m referencia al objeto 1.	—
new Test()	Comienza la construcción del objeto 2. Entra a Test(). Su campo value empieza en null.	—
super() implícito	Llama a Object(), regresa y continúa la inicialización del objeto 2.	—
String value = "A";	Objeto 2: null → "A".	—
Primer bloque: value += "B";	Objeto 2: "A" → "AB".	—
Segundo bloque: value += "C";	Objeto 2: "AB" → "ABC".	—
Cuerpo de Test(): value += "X";	Objeto 2: "ABC" → "ABCX".	—
Se completa m = new Test();	Regresa a main. m ahora referencia al objeto 2.	—
System.out.println(m.value);	Lee value del objeto 2: "ABCX".	ABCX

Cada new crea un objeto con sus propios campos. Los campos y bloques de instancia se inicializan en orden de aparición antes del cuerpo del constructor seleccionado. m = new

Test() cambia la referencia de m; no combina objetos ni ejecuta ambos constructores de Test sobre el mismo objeto.

13° ¿Cuál será el resultado de compilar y ejecutar el siguiente fragmento de código en Java?

R = c.Static block 1

Static block 2

Instance block 1

Instance block 2

Constructor

Instance block 1

Instance block 2

Constructor

Los bloques static se ejecutan una vez, en orden. Por cada new, los bloques de instancia se ejecutan en orden antes del cuerpo del constructor, sin importar dónde esté escrito el constructor.

